

Relatório — Fase 4 (concluída e validada): Verificação / pós-condições (Inserção B)

Projeto: LangNet · caso-teste ClinIA · **Data:** 2026-08-13 · **Executor:** Claude **Commit selo:** 47e697f · **Checkpoint de rollback (antes da fase):** a3bab30

A Fase 4 dá a cada task de persistência **checks declarativos** (pré e pós-condições) derivados do schema: **require_inputs** (FKs de contexto obrigatórias), **row_check** (a linha criada liga ao contexto CERTO — *differential*), **output_has** (campos da saída). Barra o "plausível mas errado" antes de persistir/avançar a cadeia. **Não altera telas.**

1. O que foi implementado

No gerador (backend/agents/langnetagents.py): - **_derive_verification(task_name, task_cfg, model)** : deriva os checks **do schema** (mínimos — não sobre-especifica, lição do SWE-bench Verified): - **require_inputs** — as FKs de contexto NOT NULL (atendimento_id / paciente_id) que o chamador deve fornecer; - **row_check** — {entity, match} : a linha criada existe e suas FKs de contexto **batem** com o input (checagem *differential*, à la PatchDiff — "passou ≠ correto"); - **output_has** — campos obrigatórios do output_schema (agêntico). - **_annotate_tasks_verification** : injeta verification: por task no tasks.yaml . - **_run_verifications** (helper nos adapters): roda as pós-condições (output_has + row_check no banco). - **Template do ws-server** (_execute_task): **pré-condição** require_inputs no início (falta FK → erro claro e cedo, sem executar/persistir); **pós-condição** após a execução (determinística e agêntica).

2. Provas de validação (saídas reais)

2.1 Derivação dos checks

```
criar_encaminhamento -> require_inputs:[atendimento_id] + row_check{encaminhamentos, match:
{atendimento_id}}
registrar_prontuario -> require_inputs:[atendimento_id, paciente_id] + row_check{prontuarios, ...}
pre_atendimento_cardiologia -> output_has:[hipoteses, nivel_confianca]
```

2.2 Teste NEGATIVO — o valor real (erro claro e cedo, sem persistir)

Chamei criar_encaminhamento **sem** atendimento_id:

```
tipo=error | erro: verificação: input(s) obrigatório(s) de contexto ausente(s)/nulo(s):
atendimento_id
| verif_falha: ['atendimento_id']
```

✓ A **pré-condição** barrou **antes** de o adapter rodar — erro **claro e cedo** (em vez do erro críptico do banco) e **nada** foi persistido.

2.3 Teste POSITIVO — E2E (`./smoke.sh`)

```
[3] ok=true (encaminhamento criado – passou pré + pós-condição)
[verify] atendimento=True | encaminhamento(medico+esp)=True | prontuario(liga_pre_diag+enc)=True
[smoke] VERDE
```

✓ No fluxo real, o `require_inputs` é satisfeito pelo *atendimento corrente* (carry) e o `row_check` **confirma** que o encaminhamento/prontuário criados ligam ao **atendimento CERTO** — regressão zero.

3. Telas

A Fase 4 não altera nenhuma tela. A mudança é nas pré/pós-condições do ws-server + no `tasks.yaml`. O smoke seguiu **VERDE**.

4. Benefício e trilha de commits

- **Benefício:** barra o "**plausível mas errado**" antes de persistir/avançar a cadeia (o filtro do SWT-Bench). Erro **claro e cedo** (pré-condição) em vez de erro críptico do banco; e a checagem **differential** (`row_check`) garante que o registro liga ao contexto correto — algo que **nem o contrato (A) nem a constraint do banco** capturam.
- **Rollback:** `a3bab30` (*ANTES da Fase 4*). **Selo:** `47e697f`.

5. Próximo passo

Fase 5 — proveniência OKF + Attested Computation (Inserção F): gravar proveniência/confiança/atualidade no vocabulário OKF v0.2 (`generated / verified / status / stale_after`) e rotular as tasks determinísticas como `Attested Computation`. Será precedida pelo **CHECKPOINT — ANTES da Fase 5**.